

DAY 3: LIGHTNING NETWORK

Instant, Cheap, Global Payments on Bitcoin

- ★ The scalability problem and why Layer 2 exists
- ★ Payment channels, invoices, and preimages
- ★ Hands-on: run your own Lightning nodes
- ★ Multi-hop routing and how it all fits together

Today: Concepts + CLI deep dive (no Python)

THE SCALABILITY PROBLEM

Bitcoin on-chain:

- ★ 2-7 transactions per second
- ★ Visa handles 65,000 TPS
- ★ A block every ~10 minutes
- ★ 1 MB (4 MW) block size limit

Hal Finney, December 2010:

"Bitcoin itself cannot scale to have every single financial transaction in the world be broadcast to everyone... there needs to be a secondary level of payment systems."

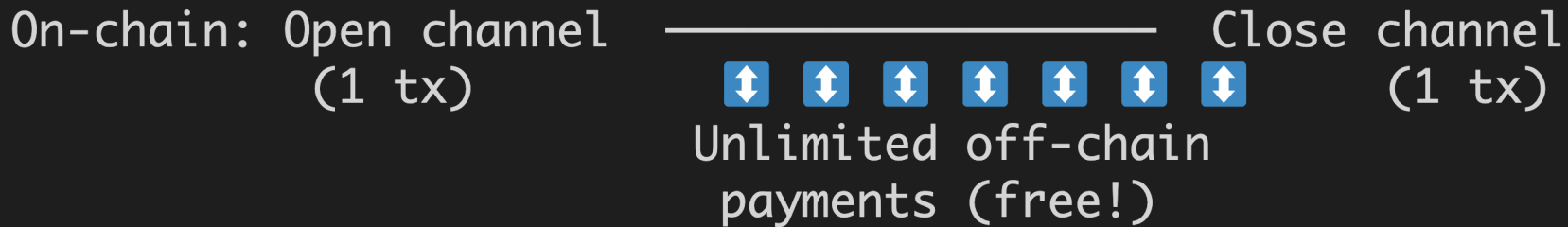
WHAT LIGHTNING SOLVES

Feature	On-chain	Lightning
Speed	10-60 min	< 1 second
Fees	\$0.50-\$50+	< \$0.01
Throughput	7 TPS	Millions TPS
Privacy	Public ledger	Onion routed
Min payment	~\$1 (dust)	1 satoshi

Lightning enables **micropayments, streaming money, and instant settlement**

THE BIG IDEA

What if two people could make unlimited payments between each other, but only broadcast the opening and closing to the blockchain?



2 on-chain transactions for **unlimited** off-chain payments

THE BAR TAB ANALOGY

Think of opening a tab at a bar:

- ★ **Open tab** = funding transaction (on-chain)
- ★ **Buy rounds all night** = Lightning payments (off-chain)
- ★ **Close out tab** = closing transaction (on-chain)

The bartender doesn't run your card for every drink.
You settle once at the end of the night.

Only 2 credit card swipes for a whole night of drinks!

HOW IT WORKS: 2-OF-2 MULTISIG

Remember **multisig** from Day 2?

- ★ A channel is a **2-of-2 multisig address**
- ★ Both Alice AND Bob must sign to spend
- ★ Neither can steal the other's funds
- ★ This is exactly what you built yesterday!



Alice's key —┐
└─ 2-of-2 Multisig Address (the channel)
Bob's key —┐

Both must agree on every state change.

Lightning Day 3

CHANNEL STATES

Each payment creates a new **commitment transaction**:

```
State 0: [Alice: 1,000,000] <—channel—> [Bob:      0]
State 1: [Alice:   700,000] <—channel—> [Bob: 300,000]
State 2: [Alice:   900,000] <—channel—> [Bob: 100,000]
State 3: [Alice:   850,000] <—channel—> [Bob: 150,000]
```

- ★ Both parties hold the **latest** commitment tx
- ★ Old states are invalidated with **revocation keys**
- ★ Only the latest state can be broadcast to the chain

WHAT IF SOMEONE CHEATS?

If Bob broadcasts an **old** state (e.g., State 1 where he had more):

- ★ Alice has the **revocation key** for State 1
- ★ Alice submits a **penalty transaction**
- ★ Alice takes **ALL** the funds in the channel
- ★ Bob loses everything

Mutual assured destruction keeps everyone honest.

The risk of losing everything > the gain from cheating.

CAPACITY VS BALANCE

Channel capacity: 1,000,000 sats (FIXED - never changes)

After open:

Alice [1,000,000 sats] $\longleftrightarrow$ [0 sats] Bob

After Alice pays Bob 300K:

Alice [700,000 sats] $\longleftrightarrow$ [300,000 sats] Bob

After Bob pays Alice 200K:

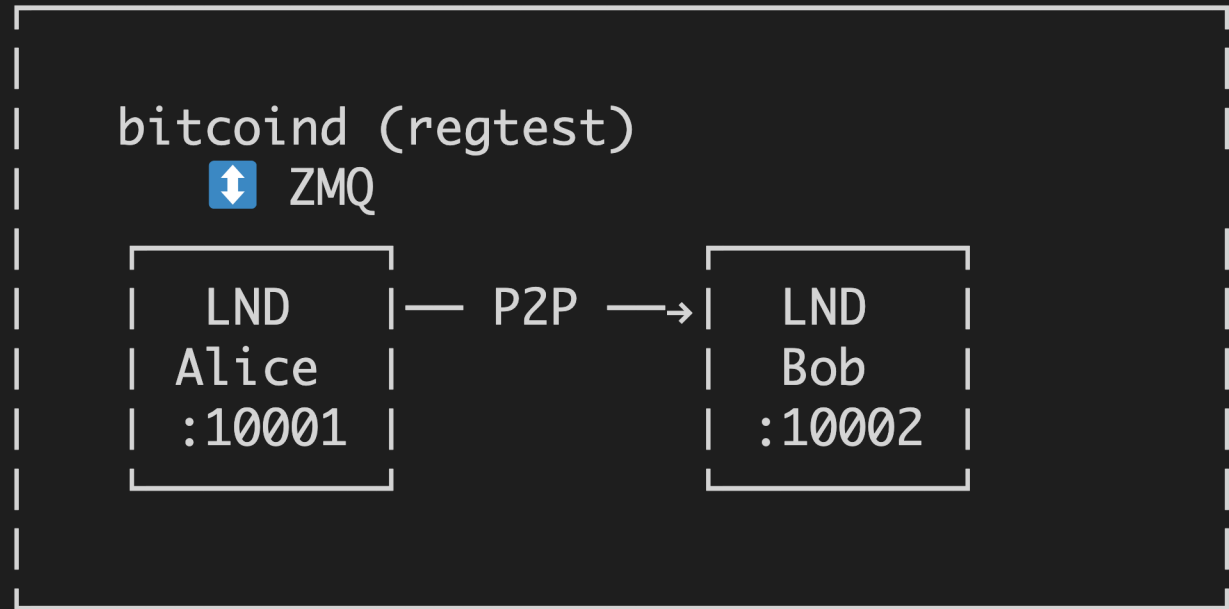
Alice [900,000 sats] $\longleftrightarrow$ [100,000 sats] Bob

★ **Capacity** = total channel size (fixed at creation)

★ **Balance** = how much each side can spend (shifts with payments)

SETUP: WHAT WE'RE BUILDING

Your Laptop



2 Lightning nodes, connected to each other, running on regtest

SETUP: INSTALL LND

Download v0.18.5 from:

`github.com/lightningnetwork/lnd/releases`

Platform	File
macOS ARM	<code>lnd-darwin-arm64-v0.18.5-beta.tar.gz</code>
macOS Intel	<code>lnd-darwin-amd64-v0.18.5-beta.tar.gz</code>
Linux 64-bit	<code>lnd-linux-amd64-v0.18.5-beta.tar.gz</code>
Windows	<code>lnd-windows-amd64-v0.18.5-beta.zip</code>

macOS/Linux: extract, copy `lnd` + `lncli` to
`/usr/local/bin/`

Windows: extract zip, put `lnd.exe` + `lncli.exe` in

Lightning Day 3

SETUP: COMPILE FROM SOURCE

If **binaries don't work** (requires Go 1.21+ from `go.dev/dl`):

```
git clone https://github.com/lightningnetwork/lnd.git
cd lnd && git checkout v0.18.5-beta
make install      # installs lnd + lncli to ~/go/bin/
```

Polar users: Add 2 LND nodes, start network, use CLI from the

Connect tab. **No drag-and-drop!** See `01_ln_setup.sh` for details.

SETUP: ZMQ + RESTART BITCOIND

Verify ZMQ in bitcoin.conf (added on Day 1):

```
zmqpubrawblock=tcp://127.0.0.1:28332  
zmqpubrawtx=tcp://127.0.0.1:28333
```

Restart bitcoind (ZMQ won't work without restart!):

```
bitcoin-cli -regtest stop  
sleep 3 && bitcoind && sleep 3  
bitcoin-cli -regtest getzmqnotifications  
# Must show pubrawblock + pubrawtx (if empty, ZMQ not configured)
```

SETUP: THREE TERMINALS

You need **3 terminal windows**:

Terminal	Purpose	Command
1	Alice's LND	<code>lnd --lnddir=./alice</code>
2	Bob's LND	<code>lnd --lnddir=./bob</code>
3	Commands	aliases below

Terminal 3 aliases (macOS/Linux):

```
alias alice="lncli --lnddir=./alice --rpcserver=localhost:10001"  
alias bob="lncli --lnddir=./bob --rpcserver=localhost:10002"
```

SETUP: WINDOWS ALIASES

Windows PowerShell:

```
function alice { lncli --lnaddr="$PWD\alice" --rpcserver=localhost:10001 @args }  
function bob { lncli --lnaddr="$PWD\bob" --rpcserver=localhost:10002 @args }
```

Windows CMD (doskey):

```
doskey alice=lncli --lnaddr=%cd%\alice --rpcserver=localhost:10001 $*  
doskey bob=lncli --lnaddr=%cd%\bob --rpcserver=localhost:10002 $*
```

Or always type the full command:

```
lncli --lnaddr=./alice --rpcserver=localhost:10001 getinfo
```

SETUP: CREATE WALLET

```
# Create Alice's wallet (interactive prompts)
alice create
# 1. Enter password (e.g. "password1234" -- regtest only!)
# 2. Confirm password
# 3. Existing cipher seed? n
# 4. Optional passphrase? (press Enter to skip)
# 5. Write down 24 words (or skip on regtest)

# Same for Bob:
bob create
```

```
# Verify both nodes are ready:
alice getinfo      # synced_to_chain: true
bob getinfo        # synced_to_chain: true
# After LND restart, unlock: alice unlock / bob unlock
```

Exercise: `bootcamp-code/day3/01_ln_setup.sh` Lightning Day 3

HANDS-ON: FUND THE WALLET

```
# Generate an LND address for Alice
alice newaddress p2wkh
# Returns: {"address": "bcrt1q..."}

# Fund from bitcoind (1 BTC)
bitcoin-cli -regtest sendtoaddress "ALICE_LND_ADDR" 1

# Mine blocks to confirm
MINER=$(bitcoin-cli -regtest -rpcwallet=alice getnewaddress)
bitcoin-cli -regtest generatetoaddress 6 "$MINER"

# Check balance (in satoshis!)
alice walletbalance
# "confirmed_balance": "100000000" (1 BTC = 100M sats)
```

HANDS-ON: CONNECT AND OPEN CHANNEL

```
# Get Bob's public key
bob getinfo      # look for "identity_pubkey"

# Connect Alice to Bob
alice connect $BOB_PUBKEY@localhost:9736
alice listpeers  # verify Bob appears

# Open a 1,000,000 sat channel
alice openchannel --node_key=$BOB_PUBKEY --local_amt=1000000
alice pendingchannels  # channel waiting for confirmations
```

```
# Mine blocks to confirm (3+ needed)
bitcoin-cli -regtest generatetoaddress 6 "$MINER"
```

```
# Check the active channel!
```

Lightning Day 3

UNDERSTANDING LISTCHANNELS

```
{  
  "active": true,  
  "remote_pubkey": "02abc...",  
  "channel_point": "txid:0",  
  "capacity": "1000000",  
  "local_balance": "996530",  
  "remote_balance": "0",  
  "commit_fee": "3470",  
  "total_satoshis_sent": "0",  
  "num_updates": "0"  
}
```

★ **capacity** = 1M sats (total, fixed)

★ **local_balance** = what Alice can send

COOPERATIVE CLOSE

Both nodes online, both agree to close:

```
alice closechannel --chan_point=TXID:INDEX  
bitcoin-cli -regtest generatetoaddress 1 "$MINER"  
alice walletbalance      # funds returned on-chain
```

Fast: 1 confirmation, low fees. Each party gets their balance back.

Exercise: `bootcamp-code/day3/02_channel_lifecycle.sh`

FORCE CLOSE

When the other party is offline or unresponsive:

```
alice closechannel --force --chan_point=TXID:INDEX  
alice pendingchannels # shows waiting_close_channels  
# Your funds locked ~144 blocks (safety period)
```

Slow: Funds timelocked to give the other party time to dispute.

If you broadcast an old state, they can take **all** your funds.

Use cooperative close whenever possible!

WHAT'S IN AN INVOICE?

```
bob addinvoice --amt=50000 --memo="Lightning Coffee"  
# Returns: {"r_hash": "abc...", "payment_request": "lnbcrt500u1p..."}
```

```
alice decodepayreq lnbcrt500u1p...
```

Field	Meaning
destination	Bob's pubkey (who gets paid)
payment_hash	SHA256(preimage) -- the lock
num_satoshis	Amount: 50,000 sats
expiry	Valid for 86,400 seconds (24h)
description	"Lightning Coffee"

THE PREIMAGE: PROOF OF PAYMENT

```
Bob:  generates random secret R (preimage)
Bob:  computes H = SHA256(R), puts H in invoice
Alice: pays -- money locked behind H
Bob:  reveals R to claim payment
Alice: keeps R as cryptographic receipt
```

After paying, verify yourself:

```
alice payinvoice lnbcrtr...
# Returns: "payment_preimage": "aabbcc..."
echo -n "PREIMAGE_HEX" | xxd -r -p | sha256sum
# Compare to payment_hash -- THEY MATCH!
```

BOLT11 FORMAT

```
lnbcrt500u1pjxyz...
```

```
| | | |
| | | └─ 500u = 500 micro-BTC = 50,000 sats
| | └─── bcrt = Bitcoin regtest
| |     (bc = mainnet, tb = testnet)
| └───── ln = Lightning Network
└──────── [encoded data + signature]
```

Amount multipliers:

- ★ **m** = milli (0.001 BTC = 100,000 sats)
- ★ **u** = micro (0.000001 BTC = 100 sats)
- ★ **n** = nano (0.00000000001 BTC)

HANDS-ON: PAY AN INVOICE

```
# 1. Bob creates invoice
bob addinvoice --amt=50000 --memo="Lightning Coffee"

# 2. Alice decodes it (inspect before paying)
alice decodepayreq lnbcrtr...

# 3. Alice pays
alice payinvoice lnbcrtr... # type "yes" to confirm

# 4. Verify preimage = proof of payment
echo -n "PREIMAGE_HEX" | xxd -r -p | sha256sum
```

HANDS-ON: CHECK PAYMENT RESULTS

```
# Check balances shifted
alice channelbalance
bob channelbalance

# Check payment history
bob listinvoices      # Bob sees settled invoice
alice listpayments    # Alice sees completed payment
```

The channel total hasn't changed -- only the split moved!

Exercise: `bootcamp-code/day3/03_invoices_deep.sh`

INVOICE EDGE CASES

```
# No-amount invoice (tip jar)
bob addinvoice --memo="Pay what you want"
alice payinvoice --amt=1000 lnbcrd...

# Short expiry (30 seconds)
bob addinvoice --amt=100 --expiry=30 --memo="Expires fast!"
# Wait 35 seconds... try to pay --> EXPIRED!

# Keysend (no invoice needed!)
alice sendpayment --dest=$BOB_PUBKEY --amt=5000 --keysend

# Pay same invoice twice --> ERROR (already settled)
# Bob pays Alice back (bidirectional!)
alice addinvoice --amt=20000 --memo="Bob pays Alice back"
bob payinvoice lnbcrd...
```

THE ROUTING PROBLEM

Alice wants to pay Charlie, but has no direct channel.

```
Alice —[channel]—> Bob —[channel]—> Charlie  
                    ???
```

How can Alice pay Charlie through Bob?

She can't just trust Bob to forward the money...

Solution: HTLCs (Hash Time-Locked Contracts)

HTLC STEP BY STEP

1. Charlie generates secret R , sends $H=SHA256(R)$ to Alice (invoice)
2. Alice → Bob:
"I'll pay you 10,001 sats IF you give me R within 100 blocks"
3. Bob → Charlie:
"I'll pay you 10,000 sats IF you give me R within 90 blocks"
4. Charlie reveals R to Bob → claims 10,000 sats
5. Bob uses R to claim from Alice → gets 10,001 sats
6. Bob keeps 1 sat as routing fee

Alice has R as proof Charlie was paid. Atomic!

TIMELOCK DECREMENTS

Alice —[100 blocks]—> Bob —[90 blocks]—> Charlie

Why does each hop get less time?

- ★ If Charlie reveals R to Bob at block 89...
- ★ Bob still has blocks 89-100 to claim from Alice
- ★ Safety margin prevents Bob from getting stuck

What if it fails?

- ★ Timelocks expire, money returns to sender
- ★ **Nobody loses funds** -- the system is trustless

HANDS-ON: ADD CHARLIE (3RD NODE)

```
# Start Charlie in Terminal 4
lnd --lnddir=./charlie

# Set up alias in Terminal 3
alias charlie="lncli --lnddir=./charlie --rpcserver=localhost:10003"
# Windows: function charlie { lncli --lnddir="$PWD\charlie" --rpcserver=localhost:10003 @args }

# Create wallet and verify
charlie create
charlie getinfo    # synced_to_chain: true
```

```
# Fund Bob (he'll open channel to Charlie)
bob newaddress p2wkh
bitcoin-cli -regtest sendtoaddress "BOB_LND_ADDR" 1
bitcoin-cli -regtest generatetoaddress 6 "$MINER"
bob walletbalance
```

HANDS-ON: OPEN ROUTE AND PAY

```
# Bob connects to Charlie and opens channel
bob connect $CHARLIE_PUBKEY@localhost:9737
bob openchannel --node_key=$CHARLIE_PUBKEY --local_amt=500000
bitcoin-cli -regtest generatetoaddress 6 "$MINER"
```

```
# Network: Alice —[1M]— Bob —[500K]— Charlie
# Verify:
alice listchannels      # Alice-Bob
bob listchannels        # Alice-Bob AND Bob-Charlie
charlie listchannels    # Bob-Charlie
```

```
# Charlie creates invoice, Alice pays through Bob!
charlie addinvoice --amt=10000 --memo="Routed payment"
alice payinvoice lnbcrt...
```

Lightning Day 3

VERIFY THE ROUTE

```
# Query route before paying
alice queryroutes --dest=$CHARLIE_PUBKEY --amt=10000
# Shows: 2 hops, fees at each, timelock decrements
```

```
# After payment, check all 3 nodes:
alice listpayments          # Total paid + routing fees
bob fwdinghistory          # Bob earned a routing fee!
charlie listinvoices        # Invoice settled
```

```
# Check channel balances on each link
alice listchannels          # Alice-Bob balance shifted
bob listchannels            # Both channels shifted
```

```
# Try too-large payment:
charlie addinvoice --amt=600000 --memo="Too large"
alice payinvoice lnbcrt... # FAILS -- insufficient capacity
```

ONION ROUTING

Lightning uses **onion routing** (like Tor):

Alice encrypts in layers:

Layer 1: [Instructions for Bob + [Layer 2]]

Layer 2: [Instructions for Charlie]

Alice → Bob: Bob peels layer 1, sees "forward to Charlie"

Bob → Charlie: Charlie peels layer 2, sees "this is for you"

- ★ Bob knows Alice paid **someone**, not necessarily Charlie
- ★ Charlie knows **someone** paid him, not necessarily Alice
- ★ Each node only knows its **previous** and **next** hop

ROUTING FEES

```
# Set routing fees on Bob's node
bob updatechanpolicy --base_fee_msat=2000 \
  --fee_rate=0.000100 --time_lock_delta=40

# Fee for a 100K sat payment:
#   Base: 2 sats + Proportional: 100K * 0.0001 = 10 sats = 12 sats total

# Route a payment with higher fees
charlie addinvoice --amt=100000 --memo="Higher fee test"
alice payinvoice lnbcrt...
```

```
# Check Bob's earnings
bob feereport
bob fwdinghistory
alice listpayments      # see the fee field
```

Exercise: [bootcamp-code/day3/05_node_management/lightning](#) Day 3

NODE OPERATIONS: BACKUP + SIGNING

```
# Channel backup (critical for real nodes!)
alice exportchanbackup --all
```

```
# Sign a message (prove node ownership)
alice signmessage --msg="I own this node"
# Returns: {"signature": "abc123..."}
```

```
# Verify the signature
alice verifymessage --msg="I own this node" --sig="SIGNATURE"
# Returns: {"valid": true, "pubkey": "..."}

```

Monitor your node

```
alice getinfo
```

```
# alias, channels, sync status
```

```
alice getnetworkinfo
```

total nodes, channels, capacity Lightning Day 3

NODE OPERATIONS: FORCE CLOSE

```
# Simulate Charlie going offline
charlie stop

# Bob force-closes the channel
bob closechannel --force --chan_point=TXID:INDEX
bob pendingchannels      # shows waiting_close_channels

# Mine blocks to complete (funds locked ~144 blocks)
bitcoin-cli -regtest generatetoaddress 150 "$MINER"
bob pendingchannels      # empty now
bob walletbalance        # funds returned on-chain
```

Why so slow? CSV timelock gives the other party time to dispute.

Lightning Day 3

REAL-WORLD ADOPTION

Lightning is live and growing:

- ★ **Pick n Pay** -- 1,628 stores across South Africa
- ★ **Strike** -- global payments and remittances
- ★ **Bitnob** -- Africa-Europe remittance corridor
- ★ **Cash App** -- Lightning send/receive for 50M+ users
- ★ **CoinCorner** -- UK-Africa payment corridor

Micropayments unlocked:

- ★ Pay-per-article, pay-per-API-call
- ★ Streaming sats (pay by the second)
- ★ Machine-to-machine payments

LIGHTNING ADDRESSES

Receiving Lightning just got simple:

★ **Lightning Address:** `alice@wallet.com` (looks like email!)

★ **LNURL:** standardized payment flows

★ **Nostr + Lightning:** zap creators with sats

Instead of: `lnbc500u1pjx7z2mpp5qg3...kf80sjqhp5d7xm...`
Just use: `alice@getalby.com`

Anyone can send you sats, anytime, no invoice needed.

DAY 3 SUMMARY

What you learned today:

- ★ Payment channels: 2-of-2 multisig, commitment txs, penalty mechanism
- ★ Channel lifecycle: fund, connect, open, use, cooperative/force close
- ★ Invoices: BOLT11 format, preimage/hash, expiry, keysend
- ★ Proof of payment: $\text{SHA256}(\text{preimage}) = \text{payment_hash}$
- ★ Multi-hop routing: HTLCs, timelocks, onion routing
- ★ Node management: fees, backups, signing, monitoring

Tomorrow: Build a Lightning webstore in Python!

HOMework

- ★ Make 10 more payments between your nodes
- ★ Try different invoice amounts and expiry times
- ★ Route 5 payments through Bob (Alice -> Charlie)
- ★ Change Bob's routing fees and observe the difference
- ★ Force close a channel, mine blocks, recover funds

Exercise scripts: `bootcamp-code/day3/`